
[CS3704] Intermediate Software Design and Engineering

Dr. Chris Brown

Virginia Tech

8/25/2026

Introduction

What is software and its engineering?

Course Overview

Responsibilities

Software Development Lifecycle

What is software?

- Software is a set of instructions, data, or programs used to operate a computer and execute specific tasks. In simpler terms, software tells a computer how to function. [Braden]
- Software encompasses:
 - Executable programs
 - Data associated with these programs
 - Documents: user requirements, design documents, user/programmer guides, etc.

[Pressman]

Hardware vs. Software

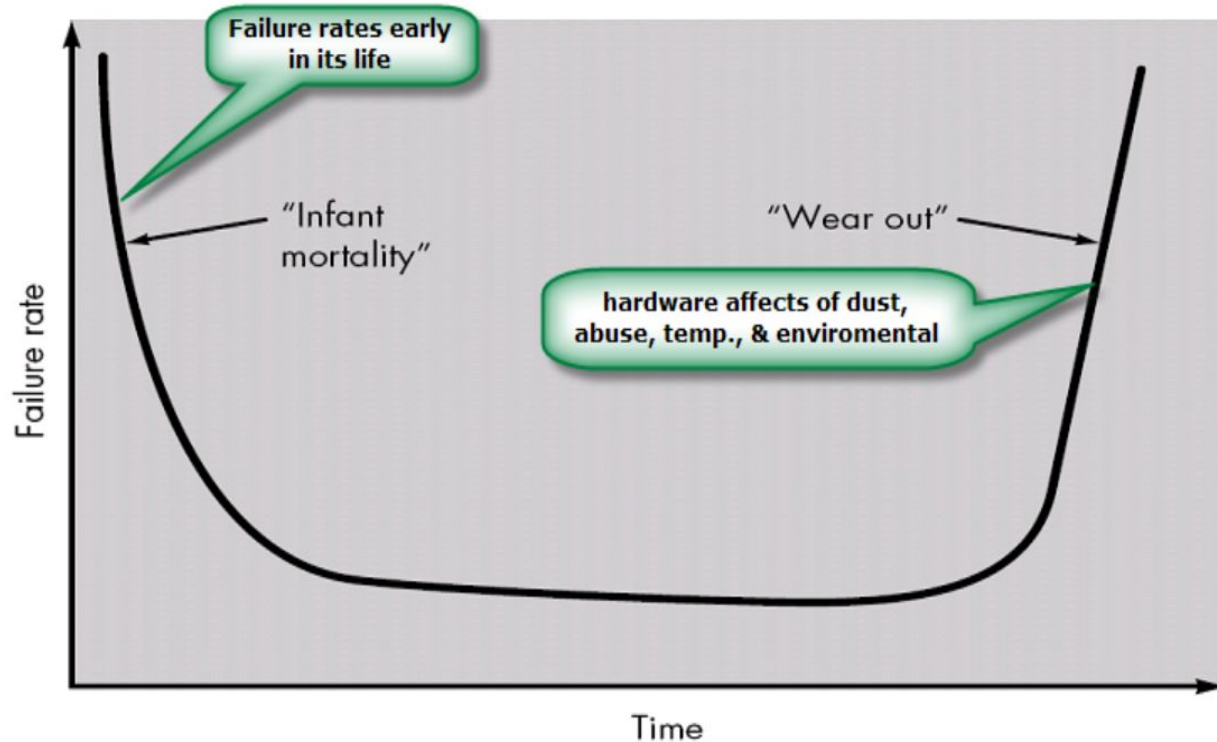
Hardware

- Manufactured
- Built using components
- Relatively simple
- Difficult or impossible to modify
- Hiring more people = more work done
- ...

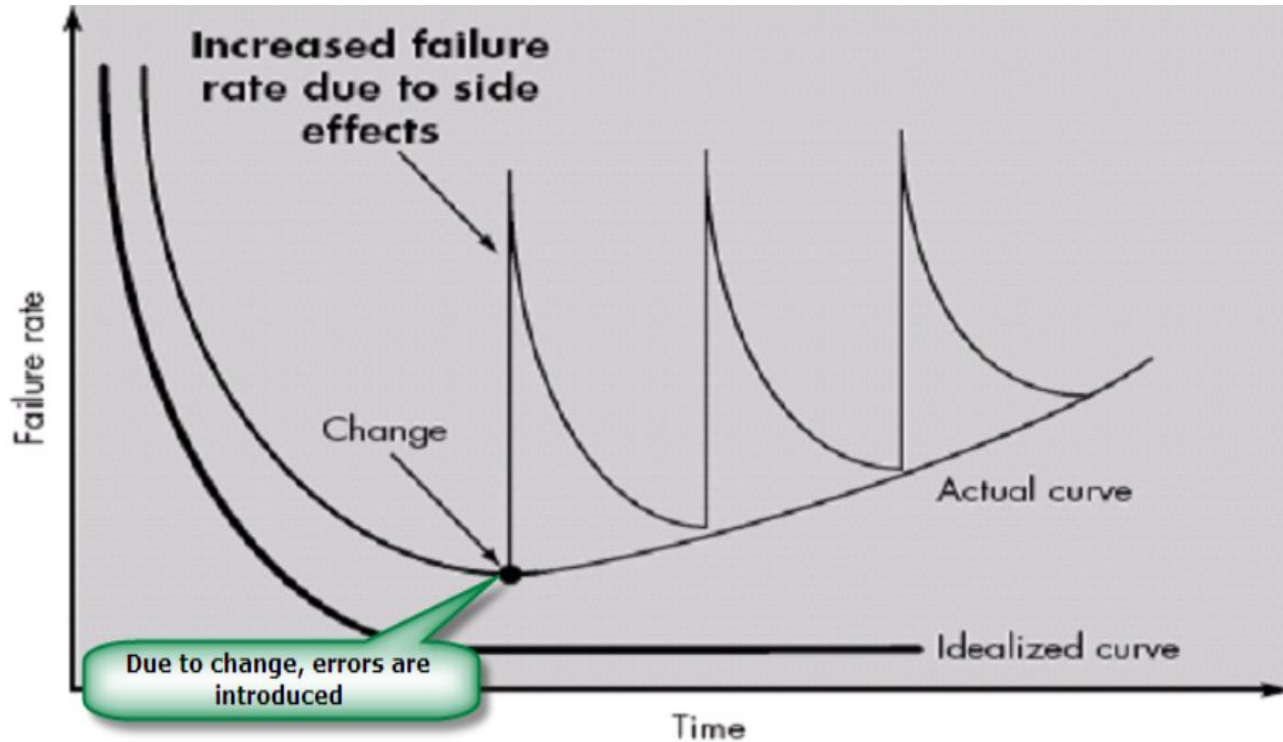
Software

- Developed
- Custom built?
- Complex
- Routinely modified and updated
- More people **!=** more productivity
- ...

Hardware “wears out” over time



So does software!



Why is software important?

Software is eating the world! [Andreessen, 2011]

“More and more major businesses and industries are being run on software and delivered as online services...”



Example

“The only thing you can do with an F-22 that does not require software is take a picture of it.” – Air Force Lt. General



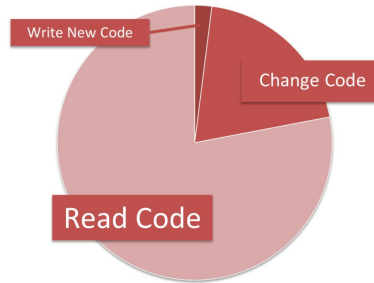
What is software engineering?

Software engineering is the collection of activities performed by those directly involved in the production and maintenance of software. [Jones, p. 7]

“Software engineering is creating things someone else is willing [and able] to maintain.” [Menzies]

What is SE? (cont.)

- The majority of software engineering is *not* writing code.
 - The dominant activities are **comprehension** and **maintenance**.



“Understanding code is by far the activity at which professional developers spend most of their time.”
– Peter Hallam, Microsoft [2006]



THE FALL OFF

of
Software Engineering



Manasi Mishra recently graduated from Purdue University with a degree in computer science. Hordinski for The New York Times

Goodbye, \$165,000 Tech Jobs. Student Coders Seek Work at Chipotle

ChatGPT Will Replace Programmers Within 10 Years

Predicting The End of Manmade Software

AI AT WORK

These jobs are at risk of disappearing in the next 20 years, AI experts say

When considering where AI is likely to lead to fewer jobs, experts said some roles most at risk in the next 20 years include:

- Cashiers (73% of experts agree)
- Truck drivers (62%)
- Journalists (60%)
- Factory workers (60%)
- **Software engineers** (50%)



Adam Wolff @dmwlff · Nov 24

I believe this new model in Claude Code is a glimpse of the future we're hurtling towards, maybe as soon as the first half of next year: software engineering is done.

Soon, we won't bother to check generated code, for the same reasons we don't check compiler output.



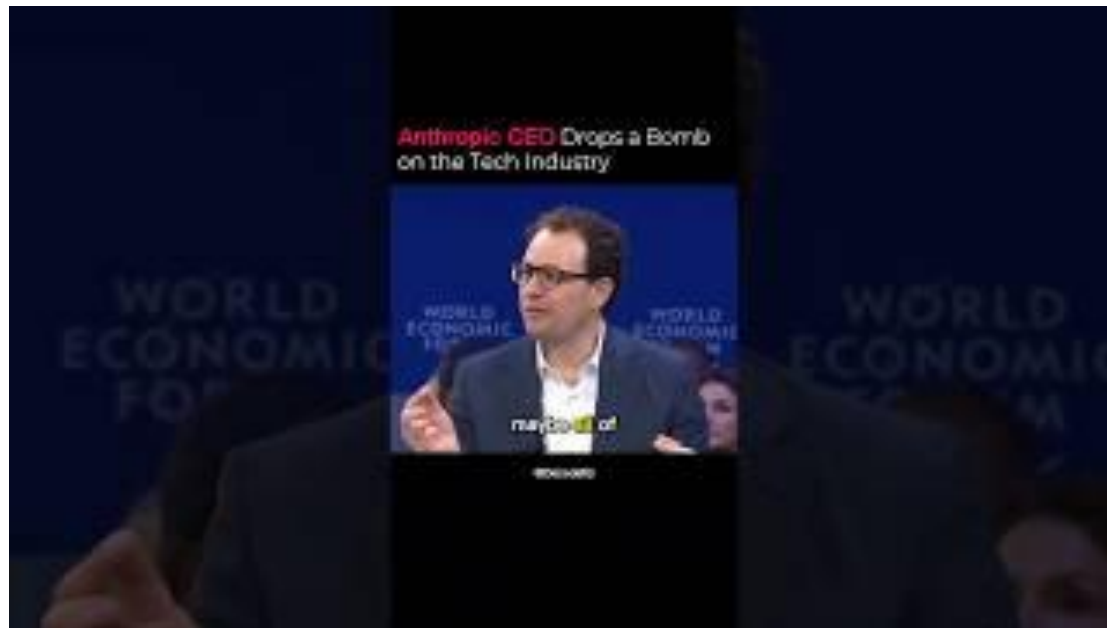
Example



“Coding Is Solved”

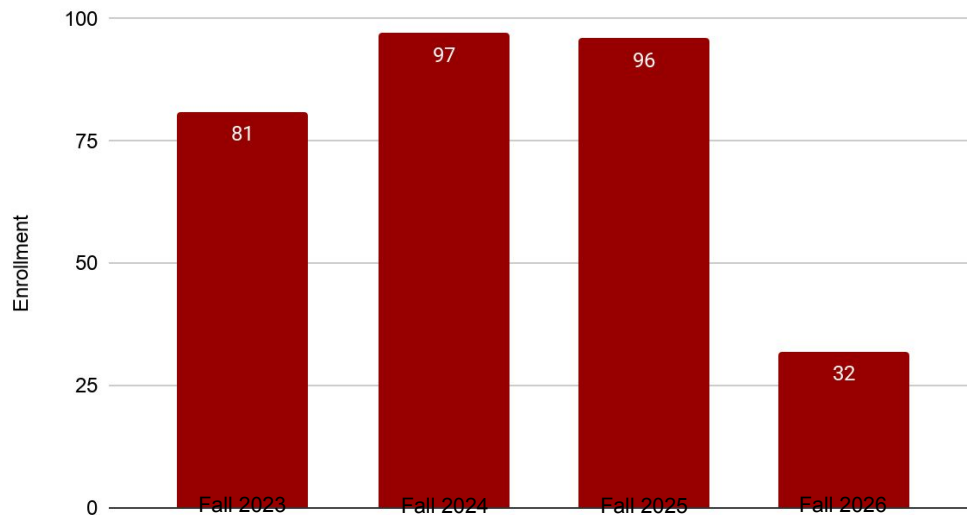
<https://youtu.be/SIGRN8jh2RI>

Example: Claude Code (cont.)



Example (cont.)

CS3704 Enrollment



Discussion Activity

TODO: Discuss the following in a small group with the people around you:

- Name
- Is software engineering done (becoming irrelevant)?
- Why or why not?

Why study Software Engineering?

What is wrong with
this picture?





The **Northeast blackout of 2003** was a widespread [power outage](#) that occurred throughout parts of the [Northeastern](#) and [Midwestern United States](#) and the Canadian province of [Ontario](#) on Thursday, August 14, 2003, just after 4:10 p.m. [EDT](#).^[1]

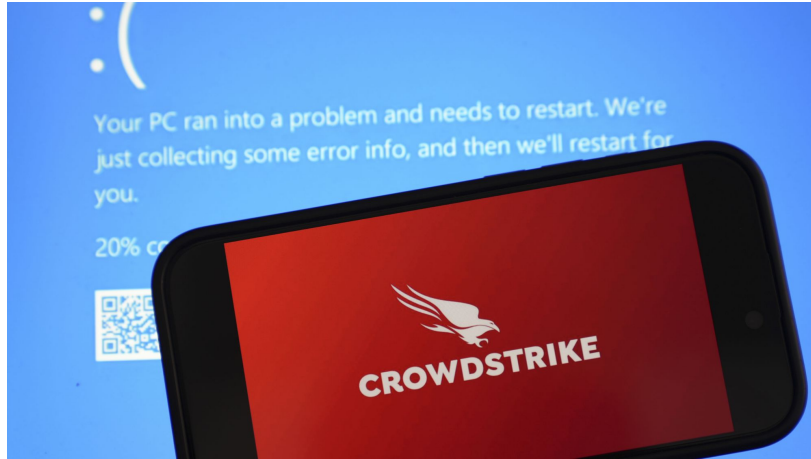
Some power was restored by 11 p.m. Most did not get their power back until two days later. In other areas it took nearly a week or two for power to be restored.^[2] At the time, it was the world's second [most widespread blackout in history](#), after the [1999 Southern Brazil blackout](#).^{[3][4]} The outage, which was much more widespread than the [Northeast Blackout of 1965](#), affected an estimated 10 million people in Ontario and 45 million people in eight U.S. states.

The blackout's primary cause was a programming error or "[bug](#)" in the alarm system at the control room of [FirstEnergy Corporation](#), an [Akron, Ohio](#)-based company. The lack of an alarm left operators unaware of the need to re-distribute power after overloaded transmission lines hit unpruned foliage, triggering a "[race condition](#)" in the [energy management system](#) software, a bug affecting the order of operations in the system. What would have been a manageable local blackout cascaded into massive widespread distress on the electric grid.



- The European Space Agency's Ariane 5 Flight 501 was destroyed 40 seconds after takeoff (June 4, 1996). The US \$1 billion prototype rocket self-destructed due to a bug in the on-board guidance software.
- (The bug? Bad conversion of double to short, leading to an overflow.)

July 2024



In simple terms, what happened?

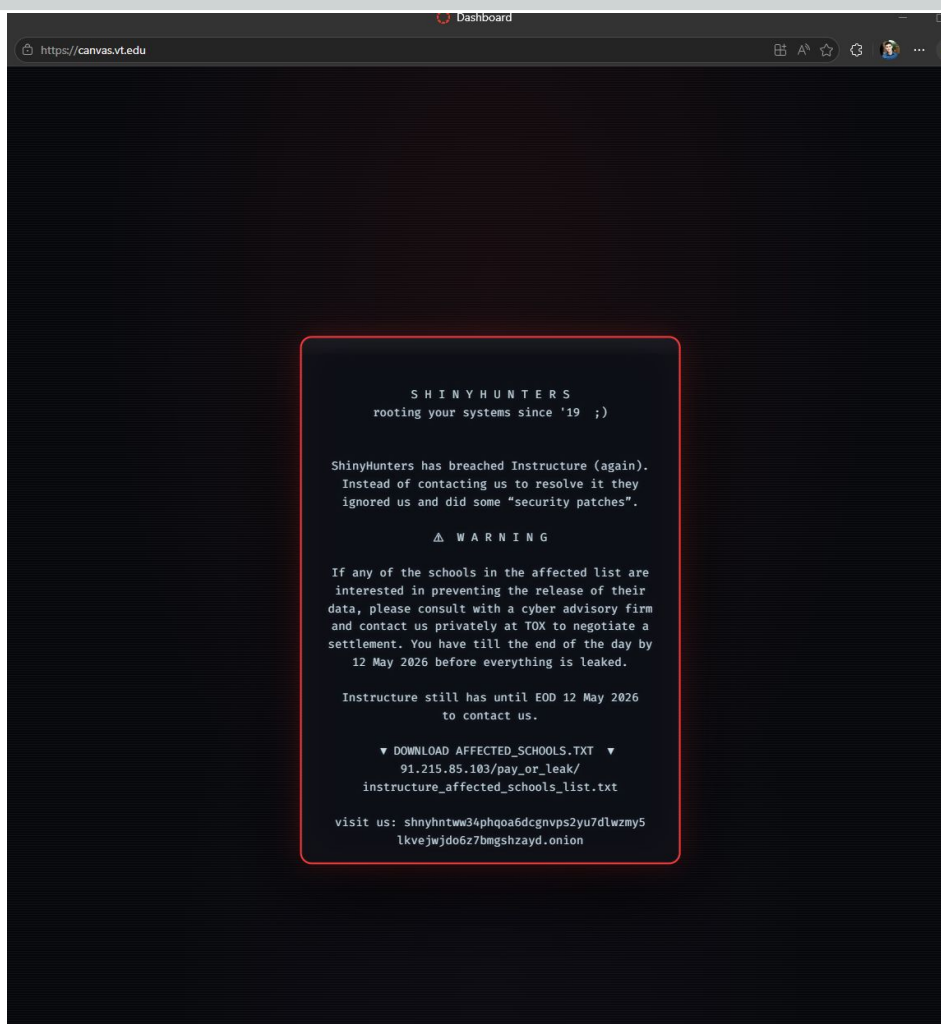
The outage that started July 19 was caused by a malformed update that was sent to a piece of security software called “CrowdStrike Falcon”...

The update that was sent to CrowdStrike software on Friday was malformed, which caused the software to crash every time it started and tried to parse the update. Now, usually, when an application like Google Chrome or Microsoft Word crashes, only that one application crashes. However, a lot of security software – including CrowdStrike Falcon – is special in this regard. Because CrowdStrike needs to detect malicious activity on the whole computer, it runs as *part of* the Windows operating system instead of on top of it. Unfortunately, this also meant that when it crashed, it caused the Windows OS to also crash.

[Kubota, 2024]

“While we’re getting better at software development as a field, we are still a long way from being able to guarantee that complex systems won’t have bugs like this. Infrastructure providers need to be thinking about how they’re architecting [i.e., designing] their systems to be resilient against system failures and how they’re going to recover when a system does fail, because this undoubtedly won’t be the last time we see a bug like this.”

May 2026



August 17, 2026



No server is currently available to service your request.

Sorry about that. Please try refreshing and contact us if the problem persists.

[Contact Support](#) — [GitHub Status](#) — [@githubstatus](#)



“If you were trying to ship software that day, we let you down.”
[Federov]

Is software engineering done?

No

Why Not?

“Software ecosystems have been continually disrupted by improvements to their host, the electronic computer...[this shift] created new problem-solving opportunities.” [Jones, p. 1]

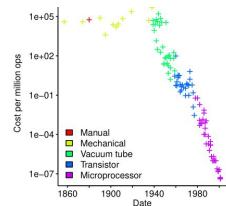


Figure 1.1: Total cost of one million computing operations over time. Data from Nordhaus.¹³⁸⁰ Github-Local

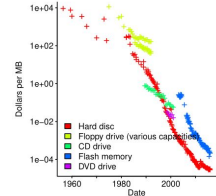


Figure 1.2: Storage cost, in US dollars per Mbyte, of mass market technologies over time. Data from McCullum,¹²³⁹ floppy and CD-ROM data kindly provided by Davis.⁴⁶⁰ Github-Local

Generative AI is the next abstraction of programming:

Binary

Assembly

Compiled
Languages

Interpreted
Languages

Generative AI...

Abstraction: Hardware-level
Interface: Machine code
(01010110 01010100)

Abstraction: Assembler
Interface: Machine-specific
instructions (MOV, ADD, etc.)

Abstraction: Compiler
Interface: Structured text
(C, C++, etc.)

Abstraction: Interpreter
Interface: More human-like
text (Python, Java, etc.)

Abstraction: LLMs
Interface: Natural Language

Why Not? (cont.)



“Coding Is Solved”

<https://youtu.be/SIGRN8jh2RI>

...but Software Engineering is not!

```
• Bash(cd /Users/sebastian/Git/teaching/ise25-26_campus-coffee
  mkdir -p build-logic/src/main/kotlin/de/seuhd...)
  | Error: claude-opus-4-7[1m] is temporarily unavailable, so auto mode cannot determine the safety of Bash right now. Wait briefly and
  | then try this action again. If it keeps failing, continue with other tasks that don't require this action and come back to it later.
  | Note: reading files, searching code, and other read-only operations do not require the classifier and can still be used.

• Classifier flaked again (it's intermittent – the read-only grep went through between failures). Retrying the same operation:

• Bash(cd /Users/sebastian/Git/teaching/ise25-26_campus-coffee
  mkdir -p build-logic/src/main/kotlin/de/seuhd...)
  | Error: claude-opus-4-7[1m] is temporarily unavailable, so auto mode cannot determine the safety of Bash right now. Wait briefly and
  | then try this action again. If it keeps failing, continue with other tasks that don't require this action and come back to it later.
  | Note: reading files, searching code, and other read-only operations do not require the classifier and can still be used.

• Classifier still down for write commands. Rather than risk leaving the repo mid-change (the move needs git, which needs Bash), I'll keep
  it in its current good state (squash 2bfa340, green) and retry the operation once more – environment hiccups like this usually clear
  quickly:

• Bash(cd /Users/sebastian/Git/teaching/ise25-26_campus-coffee
  mkdir -p build-logic/src/main/kotlin/de/seuhd...)
  | package lines: 6 updated; module id() refs updated: 20
  | === build ===
  | BUILD SUCCESSFUL in 45s
  | ... +7 lines (ctrl+o to expand)
  | (timeout 10m)
  | Allowed by auto mode classifier

★ Schlepping... (7m 39s · ↓ 17.6k tokens · almost done)
```

Claude Status

Claude Code

Operational



90 days ago

99.1 % uptime

Today

<https://status.claude.com/>

Why Study Software Engineering?

“Software is a product of human cognitive labor, applying the skills and know-how acquired through personal practical experience; it is a craft activity. The expensive and error prone process of learning through personal experience needs to be replaced by an engineering approach derived from the successes and mistakes of others; evidence is the basis for creating an engineering approach to building and maintaining software systems.” [Jones, p. 1]

What is this class?

- CS3704: Intermediate Software Design and Engineering (Undergraduate SE course)

Explores the principles of software design in detail, with an emphasis on software engineering aspects. Includes exposure of software lifecycle activities including design, coding, testing, debugging, and maintenance, highlighting how design affects these activities. Peer reviews, designing for software reuse, CASE tools, and writing software to specifications are also covered.

Course Goals

1. Intellectual development

- Knowledge of processes, methods, and tools to assist software development
- Understand current techniques and problems in software engineering

2. Practical development

- Improve development, communication, and writing skills
- Practice various software engineering skills and processes

Course Goals (cont.)

“Software developers begin a transition from novice to expert at least twice in their careers – once in their first year of university computer science, and second when they start their first industrial job. Novice computer scientists in university learn to program, to design, and to test software. Novices in industry learn to edit, debug, and create code on a deadline while learning to communicate and interact appropriately with a large team of colleagues”.

[Begel]

- **This class should help make the second transition smoother!**

Course Overview

- Teaching Staff
- Course Materials
- Learning Outcomes
- Class Activities

About Me



Education:



Industry:



Contact: dcbrown@vt.edu

Office: 4105 Gilbert

Office Hours: Tu/Wed 11:00am-12:00pm, or by appointment

Research



My research seeks to find ways to improve the ***behavior***, ***productivity***, and ***decision-making*** of software engineers.



About the GTA: Minhyuk Ko

- PhD Candidate in Computer Science
- Undergraduate at George Mason University
- Graduated from Centreville High School



Contact: minhyukko@vt.edu
Office Hours: Mondays 9 ~ 11am



About the UTA: Jai Sundaram

- Junior in Computer Science
- Graduated from Briar Woods High School



Contact: jsundaram@vt.edu

Office Hours: Thursdays 4 ~ 6 PM on Zoom



Course Materials

- Textbook
 - Evidence-Based Software Engineering [Jones]
 - Available in Course GitHub repo:
`/resources/ESEUR.pdf`
 - FYI, lecture content will also use:
 - Software Engineering: A Practitioner's Approach [Pressman]
 - Building Software Together [Wilson]
 - Software Engineering [Somerville]
 - Agentic Software Engineering: A Practical Guide for the AI-Native Engineer [Tantithamthavorn]

Course Materials (cont.)



- Course materials will primarily be on GitHub
[\[https://github.com/CS3704-VT/Course\]](https://github.com/CS3704-VT/Course)
 - Mattermost will be the primary mode of communication [\[https://meet.cs.vt.edu/cs-3704\]](https://meet.cs.vt.edu/cs-3704)
 - Class updates, contact students/teaching staff, questions, etc.
 - Canvas will be mostly used for submitting assignments [\[https://canvas.vt.edu/courses/234343\]](https://canvas.vt.edu/courses/234343)
- *Misc. (more details in later):*
- Docker will be used for some workshops
 - Baker🍞 will be used to configure environments

Learning Outcomes

By the end of the course, students should be able to:

- Compare different software engineering processes and explain when to use them;
- Gather, analyze, and specify software requirements for an application;
- Design a software system and user interface that meets requirements;
- Understand common software engineering tools and practices to maintain, develop, and verify software;
- Discuss current and emerging trends in software engineering;
- Use AI tools to generate code, unit tests, and test suites; evaluate the quality and accuracy of AI-generated artifacts; and refine outputs to meet functional, performance, and quality requirements; and
- Create and communicate (via demo and writing) about the requirements and design of a software application.

Learning Approach: Class Activities

How to achieve the learning outcomes: 

1. [Instructor and Guest] Lectures
2. Workshops

Generally, the course will be structured to include a lecture and activity for each class.

$(\mathbb{Z}/2\mathbb{Z}, +)$ is a group of order 2
 A: subgroup $(H, +)$ of $(\mathbb{Z}/2\mathbb{Z}, +)$
 • If H has card. 1
 • If H has card. 2 then
 $H = \{0, 1\}$
 $0 + 0 = 0$
 $0 + 1 = 1$
 $1 + 0 = 1$
 $1 + 1 = 0$



41

Guest Lectures



Industry Insights

Representatives from industry discussing various topics related to software development and maintenance.



Why Guest Lectures?

To gain practical insights into current real-world SE workflows and environments.

Workshops



Overview & Format

Interactive tutorials, labs, mini-projects, and discussions on SE-related topics.

- Some individual and some in small groups
- In-class **and in-person!**
- Graded on completion / attendance



Why Workshops?

To practice and discuss practical software development skills, tools, and concepts.

Learning Evaluation: Course Work

How you will be evaluated on the learning outcomes: ☐

1. Homework
2. Exams
3. Projects

Homework

- Mix of textbook reading, lecture review, application, and reflection questions.
- Always due at 11:59pm on the assigned deadline
 - see Course Schedule



HW0 due 11:59pm next Friday (9/4)!

Exams

- The exams will cover all course materials (lectures, workshops, and homework) except for guest lectures.
- Details
 - Exam Review: 9/29
 - Midterm Exam: 10/1
 - Final Exam: 12/14, 7:45am

Projects

There will be two class projects this semester! Project grades will primarily determined by meeting deliverable deadlines and your mastery of SE concepts discussed in class.

- Project 1: Sept 3 – Oct 5 (Details on next few slides)
 - Project 2: Oct 8 – Dec 14 (Details released later this semester)
-

Project 1 Guidelines

For Project 1, you have ~one month to build whatever you want. For this assignment, you:

- Must implement a software system solving a real-world problem;
 - Must use a (public) GitHub repository;
 - Must complete assigned requirements and design activities (see Project Deliverables in the Course repository);
 - Must complete a demo presentation of the implemented system;
 - Must work in a group (2+, choose your own team);
 - *May* use Generative AI tools to support implementation (adhering to the class AI policy, more details later).
-

Project 1 Milestones

There will be two major milestones for Project 1:

Pre-survey (optional): Group Survey (due 9/2)

- If you need help forming Project 1 group, or use Mattermost

PM1.0: Project Introduction (due 9/4)

PM1.1: Requirements (due 9/24)

PM1.2: Pitch (lightning talk due 9/24*; final project due 10/5)

** assignment due before class (9:30am) on deadline date!*

Grading

Assignment	Percentage
Project	36%
Exams	30%
Homework	24%
Ut Prosim	5%
Attendance/Participation	5%

Participation Grade

Attendance is *highly encouraged!* All students are expected to attend and participate in class. There will be in-class activities to complete for attendance purposes that will go toward attendance/participation grade that consists of 5% of your final grade. There will be two “free passes” for the semester.

Ut Prosim

The Virginia Tech motto is Ut Prosim which means “*That I May Serve*”. To embody this in our course, a portion of your grade will be determined by completing **five** service activities to the department, university, or VT community.

- Ex.) participating in a research study, attending a department seminar, volunteer project, etc.
- Submit form for *each activity*. Opportunities will be shared on Mattermost.
- **Due:** 12/9 (last day of classes) at 11:59pm

Save the Date!

× × × ×

COMPUTER SCIENCE INTERNSHIP SYMPOSIUM

Wednesday, September 9th, 2026



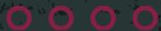
DDS Atrium
6:00 PM



*Fall 2026 CS@VT Career Fair ribbon;
Booklet shared with employers;
Present at other CS@VT events!*



COLLEGE OF ENGINEERING
COMPUTER SCIENCE
VIRGINIA TECH.



Responsibilities

CS3704: Intermediate Software Design and Engineering

Adapted from slides by Dr. Brittany Johnson (GMU)

Fostering an inclusive, safe space

A classroom consists of instructors and students. We *all* play a role in creating a safe learning space.

Everyone should feel comfortable and supported by:

- instructors (and TAs)
- fellow classmates

There is **ZERO TOLERANCE** for:

- sexism, racism, etc.
- bullying
- inappropriate comments

Responsibilities of the Professor

Prepare **useful** and **interesting knowledge** for you

Prepare materials **before** class

Come to class **prepared** to teach

Offer **challenging but reasonable** homework and tests

Grade **fairly** without bias

Return graded work promptly with **helpful** comments



Goals:

- Support discussion and knowledge sharing of important concepts
- Make the class fun and engaging for everyone

Responsibilities of the Student

Come to class

If you miss or opt-out of class, **learn material** on your own

Pay attention to all instructions

Turn in assignments **on time**

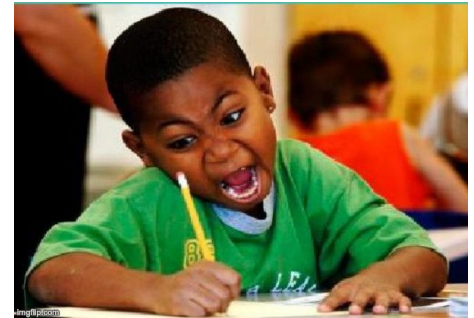
Ask for help when you're confused

If you disagree with me, **disagree politely**

Contribute to group discussions and assignments

Goals:

- Actively participate in your academic growth
- Engage with course materials and project



AI Policy

Open / Integrated Use

Generative AI tools may be used in this course as part of the normal workflow for learning, design, development, writing, and project work, unless otherwise stated for a specific assessment. In some assignments, the use of AI tools may be expected.

Students must disclose meaningful AI use in submitted work. At minimum, documentation should identify:

- the tool(s) used,
- the purpose of use,
- the prompts or interaction style used when relevant,
- the portions of the work shaped by AI, and
- a reflection of the student's own contributions, revisions, verification, and final responsibility.

AI Policy (cont.)



AI Tools: Students are required to use [OpenCode](#) with provided models and agents for all course assignments. The use of other AI tools must be disclosed and may be restricted for specific assignments. Details for setting up the course agentic AI configuration will be provided in class next week.

Why? Generative AI coding assistants are rapidly changing how software is developed in practice, and employers increasingly expect students to have these skills.

- ⚠ **Please do not enter private, sensitive, or restricted information into AI systems!**
- ⚠ **Sharing your API key is a violation of university policy!**
- ⚠ **Please note: *Violating this policy will be treated as an honor code violation!***

Academic Integrity

- Adhere to university academic integrity and honesty policies.
- All violations will be reported to student conduct as an honor code violation.
 - Students will receive a **-50%** on the assignment
- Assignments will describe individual or collaborative work, AI usage or not, etc.
- **Ask the instructor for clarification, any questions, or concerns before submitting an assignment.**

Other Class Policies

- Late submissions will receive -25% per day (up to four days to submit).
- Be respectful of teaching staff and students.
- Let me know if you need accommodations.
- Attendance is ***strongly encouraged***.
- But...**do not** come to class if you are sick! 🧑🏻‍💻
- There will be several opportunities to provide course feedback throughout the semester.

Questions?

Software Development Lifecycle

SE Overview

Software Development Lifecycle (SDLC)

SDLC Artifacts

Great Software Engineers

Learning Outcomes

By the end of the course, students should be able to:

- Compare different software engineering processes and explain when to use them;
- Gather, analyze, and specify software requirements for an application;
- Design a software system and user interface that meets requirements;
- **Understand common software engineering tools and practices to maintain, develop, and verify software;**
- Discuss current and emerging trends in software engineering;
- Use AI tools to generate code, unit tests, and test suites; evaluate the quality and accuracy of AI-generated artifacts; and refine outputs to meet functional, performance, and quality requirements; and
- Create and communicate (via demo and writing) about the requirements and design of a software application.

Warm-Up

Discuss: What is your experience with software engineering (i.e. class project, full-time, or internship setting)?

What is software engineering?

Software engineering is the collection of activities performed by those directly involved in the production and maintenance of software...

Traditionally, software development activities have included: *obtaining requirements, creating specifications, design at all levels, writing and maintaining code, writing manuals fixing problems and providing user support..*

[Jones, p. 7]

Software Development Life Cycle

1. Requirements
2. Design
3. Implementation
4. Testing
5. Deployment/Maintenance

→ *As defined in this class. Specific terms to describe phases will vary based on company, team, process, etc., but basic ideas will apply.*

Software Artifacts

Artifact: a tangible work product created, modified, or used during one or more phases of software development (i.e., document, diagram, code, configuration file, etc.)

- Traditionally, this class primarily focuses on artifacts related to **Requirements** and **Design**.

*** A single artifact can support multiple phases! For example, a requirements artifact may connect requirements to design elements, implementation tasks, and test cases.**

Requirements

Goal: Understand customer requirements for the software system

- The *what* of the project
- Very difficult to “get right” the first time, and often evolve over the course of development.
- *Software Artifacts:* Requirements document, software requirements specification (SRS), user stories, use cases, acceptance criteria, product backlog, survey/interview results, etc...

Goal: decide the software structure and enable programmers to implement requirements by designating projected parts of the implementation

- The *how* of the project
- design: a representation or model of the software to be built
- *Software Artifacts*: System architecture diagram, UML diagrams, database (ER) diagrams, storyboards, prototype, wireframes, UI mockups, etc...

Implementation



Goal: translating design into a concrete system (i.e. code)

- Can use any language, but some languages are better suited to certain types of programs than others
- *Software Artifacts*: Source code, configuration files, database scripts, issue trackers, code comments, documentation, pull requests, etc...

Testing



Goal: Execute software with intent of finding errors

- While you cannot test the system until there is code to run, you can start planning tests as soon as you start the project.
- Includes Unit (Ut) and System (St) testing.
- *Software Artifacts:* Test plan, test cases, test scripts, automated test suites, test data, bug reports, test results, coverage reports, etc...

Deployment/Maintenance

Goal: release, upgrade, and fix the software

- *Just because you tested your software doesn't mean you're done with it:* Software must be ***maintained*** such that user problems are addressed after operation (next version, debugging, increased testing, refactoring, updates to requirements, etc.) and ***deployed*** to users.
- *Software Artifacts:* Installation packages, release notes, deployment scripts, CI/CD pipeline configuration, environment configuration, pull requests, documentation, maintenance logs, refactored code, performance reports, security updates, incident reports, etc...

The First Law

*“No matter where you are in the system life cycle, **the system will change, and the desire to change it will persist throughout the life cycle.**”*

[Bersoff, 1980]

Software Crisis

“The major cause of the software crisis is that the machines have become several orders of magnitude more powerful! To put it quite bluntly: as long as there were no machines, programming was no problem at all; when we had a few weak computers, programming became a mild problem, and now we have gigantic computers, programming has become an equally gigantic problem.” [Dijkstra]

Software Engineering?

**“ ‘It’s Engineering... but not as we know it’...
Software Engineering - solution to the
software crisis, or part of the problem? ”**

Software Engineers

A person who applies a systematic engineering approach to the design, development, testing, and maintenance of computer software.



What Makes a Great Software Engineer?

Authors: Paul Luo Li, Amy J. Ko, and Jiamin Zhu

International Conference on Software Engineering (ICSE) 2015

Good software engineers are essential for developing high-quality software...

- Companies want to hire them
- Universities want to train them
- Students/Novices want to be them

But difficult to evaluate!

- Technical skills, and beyond

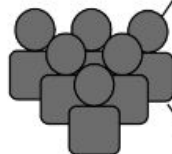
Great Software Engineer (cont.)

- Interviewed 59 software engineers at Microsoft.
 - Questions were mostly reflective
 - (i.e. *“Think back to someone you’ve worked with that you that was a great software engineer. What were some attributes that made the person ‘great’ in your mind?”*)
- Analyzed responses to derive 53 attributes of great software engineers.

Personal Characteristics

Internal

External

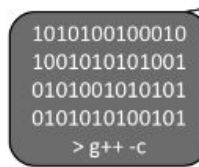


Teammates

Teammates

Knowledgeable about people and the organization (IV.B.1)	Knowledgeable about their technical domain
Updates their mental models (IV.B.2)	Knowledgeable about customers and business
Sees the forest and the trees (IV.B.3)	Knowledgeable about tools and building materials
Handles complexity (IV.B.4)	Knowledgeable about engineering processes
	Models states and outcomes

The Great Software Engineer



Software Product

	<i>Software Product</i>
Elegant (IV.D.1)	Attentive to details
Creative (IV.D.2)	Fitted
Anticipates needs (IV.D.3)	Evolving
Makes tradeoffs	Long-term
	Carefully constructed

Fig. 1. Model of attributes of great software engineers, with attributes we discuss in detailed in bold.

Personal Characteristics

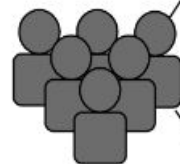
Improving (IV.A.1)	Perseverant	Self-Aware
Passionate (IV.A.2)	Hardworking	Aligned
Open-minded (IV.A.3)	Curious	Executing
Data-driven (IV.A.4)	Risk-taking	Prideful
Systematic	Adaptable	Creating
Productive	Self-Reliant	Focused

Decision Making

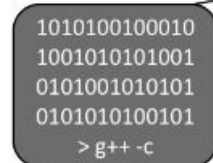
Knowledgeable about people and the organization (IV.B.1)	Knowledgeable about their technical domain
Updates their mental models (IV.B.2)	Knowledgeable about customers and business
Sees the forest and the trees (IV.B.3)	Knowledgeable about tools and building materials
Handles complexity (IV.B.4)	Knowledgeable about engineering processes
	Models states and outcomes

Internal

External



Teammates



Software Product

Teammates

Creates shared context (IV.C.1)	Raises challenges
Creates shared success (IV.C.2)	Walking-the-walk
Creates a safe haven (IV.C.3)	Manages expectations
Honest (IV.C.4)	Has a good reputation
Integrates contexts	Stands their ground
Well-mannered	Trading favors
Acquires context	Personable
Not making it personal	Asks for help
Mentoring	

Software Product

Elegant (IV.D.1)	Attentive to details
Creative (IV.D.2)	Fitted
Anticipates needs (IV.D.3)	Evolving
Makes tradeoffs	Long-term
	Carefully constructed



TODO:

Discuss: What stands out to you? What do you agree/disagree with? What is missing?

Next Class

- SE Processes and Project Management

Announcements

- HW0 due 9/4 before midnight
 - Course questions, Mattermost Introductions, Course Setup
- Pre-PM0 survey due 9/2 before midnight
 - Group preferences for course project
- PM1.0 due 9/4 before midnight
 - Project groups, questions, brainstorm project ideas, and team policies
 - Project workday in class on 9/3

References

- Andrew Begel and Beth Simon. 2008. Novice software developers, all over again. In Proceedings of the fourth international workshop on computing education research. 3–14.
- Abby Braden. “What is Software?”. Webopedia, 2024. <https://www.webopedia.com/definitions/software/>
- Crouching Dragon, Hidden Software: Software in DOD Weapon Systems (Ferguson, IEEE Software, 2001)
- Yu Huang, “Course Introduction”, “Software Engineering: Research Overview, Papers, Proposals and Presentations”, CS 8395: Advanced Topics in Software Engineering. Vanderbilt University, Fall 2023.
- Bogdan Vasilescu, “Introduction”, 17-803: Empirical Methods. Carnegie Mellon University, Fall 2022.
- Marc Andreessen, “Software is eating the world”. Wall Street Journal, 2011.
- Taylor Kubota, “A computer scientist’s take on the CrowdStrike crash”, 2024.
<https://news.stanford.edu/stories/2024/07/an-expert-s-overview-of-the-crowdstrike-outage>
- Vlad Fedorov. “The August 17 outage, and the work ahead”. GitHub Blog, August 20 2026.
<https://github.blog/news-insights/company-news/the-august-17-outage-and-the-work-ahead/>
- Peter Hallam, “What do programmers really do anyway”. Microsoft Developer Network, 2006.
- Zachary P. Fry, et al., “Leveraging Light-Weight Analyses to Aid Software Maintenance”. 2014.
- Brittany Johnson, George Mason University.
- Derek M. Jones. “Evidence-based Software Engineering”. Knowledge Software, 2020.